

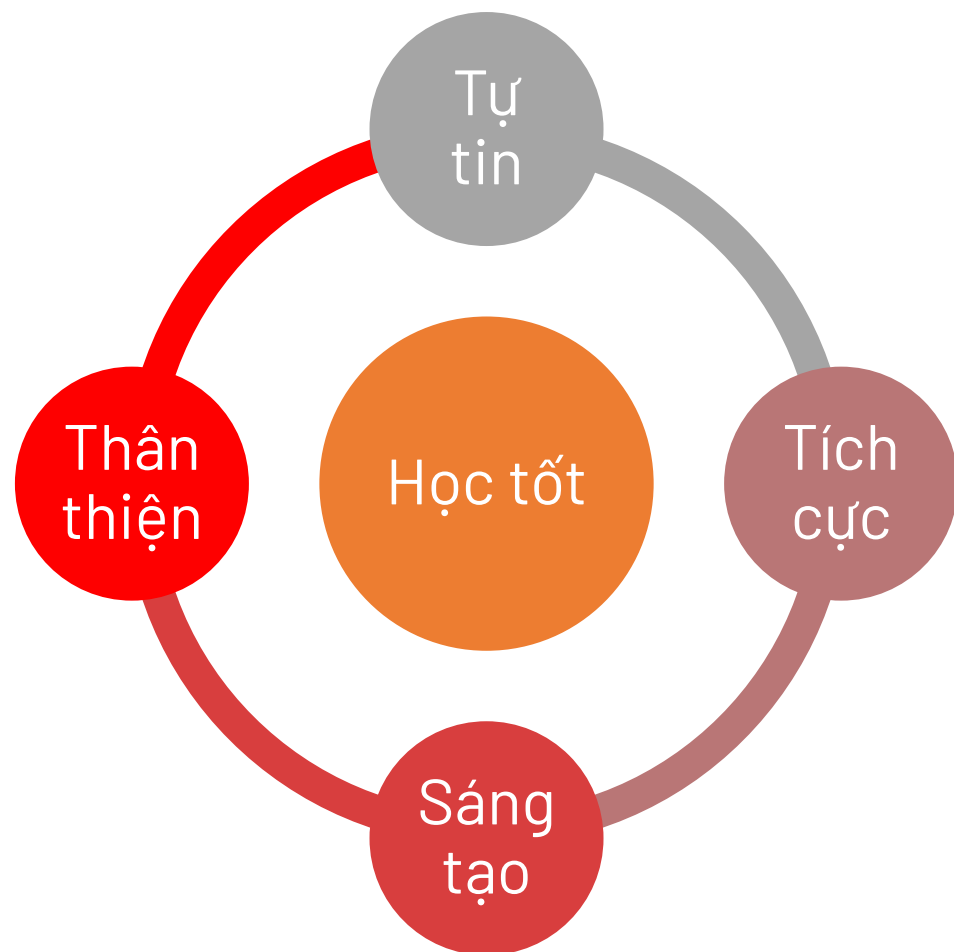


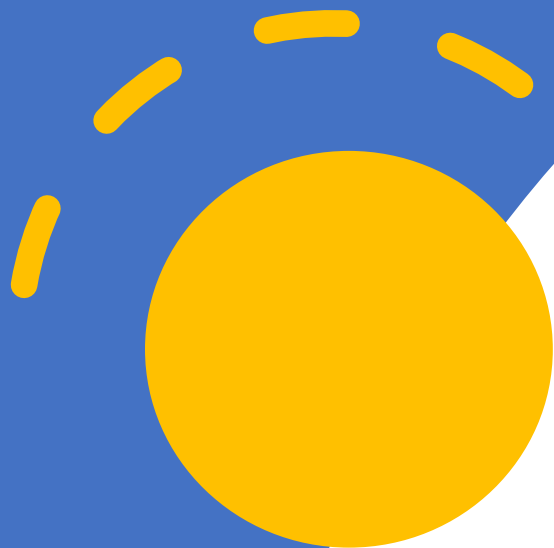
TRUNG TÂM ĐÀO TẠO CÔNG NGHỆ TIPY STEM
www.tipy.edu.vn – hotline: 084 3537 333





PHẦN THƯỞNG HỌC TỐT





DỰ ÁN

*Lập trình phần mềm đồ họa
Paint với Python*



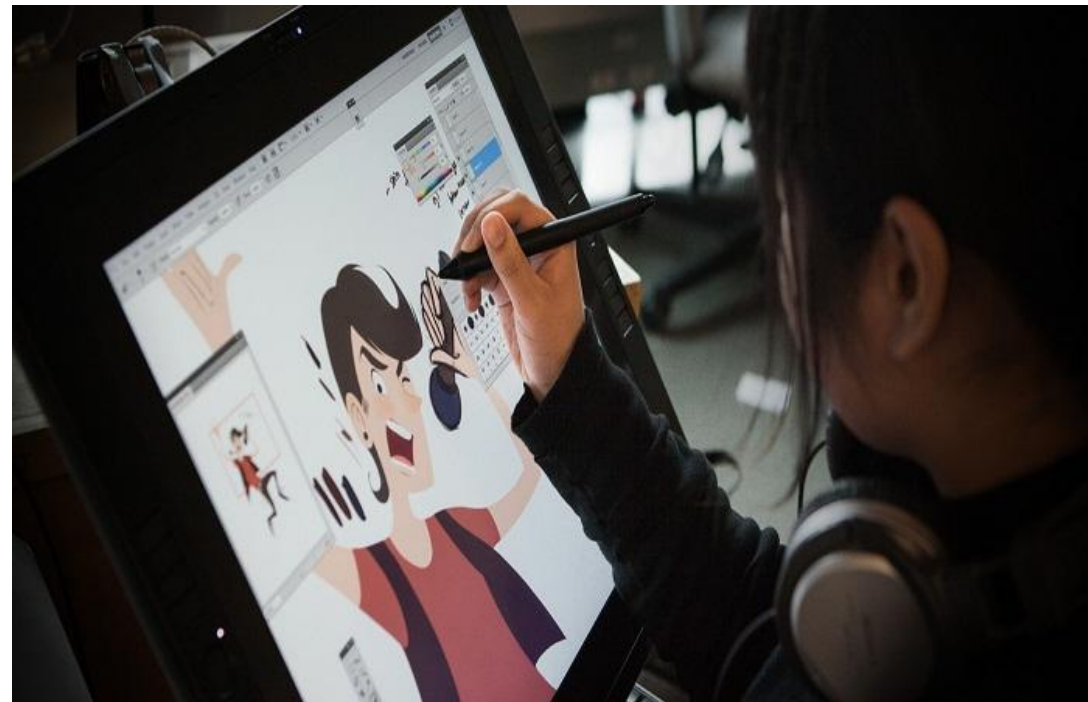
KHÁM PHÁ BÀI HỌC

Tại sao họa sĩ hiện đại thích vẽ trên máy tính ?

Hàng ngàn năm qua, con người vẽ trên vách đá, trên lá, rồi trên giấy. Nhưng tất cả đều có một nhược điểm: **Vẽ sai là bỏ đi!**

Phần mềm đồ họa ra đời mang đến những thay đổi toàn diện:

1. Cục tẩy thần kỳ không bao giờ làm rách giấy.
2. Pha trộn hàng triệu màu sắc mà không bị lem.
3. Lưu trữ và chia sẻ cho hàng triệu người trong 1 giây.



KHÁM PHÁ BÀI HỌC

Mọi tựa game **nổi tiếng** (Minecraft, Roblox, Liên Quân) hay các bộ phim hoạt hình đều bắt đầu từ... **một nét vẽ trên màn hình**.

- Các **phần mềm đồ họa** (như **MS Paint**, **Photoshop**) chính là nơi sản xuất ra thế giới ảo.

Việc hiểu cách phần mềm vẽ hoạt động là bước đầu tiên để trở thành:

- *Nhà thiết kế nhân vật Game (Game Artist).*
- *Họa sĩ minh họa kỹ thuật số (Digital Artist).*



Cửa sổ
ứng dụng

Tiêu đề

Kích thước

Đối tượng

Canvas

Frame

Button

Scale

Thư viện

Tkinter

Colorchooser

FileDialog

Mô tả
Ứng dụng

Người dùng vẽ tự
do bằng chuột.

Thay đổi màu sắc,
tẩy xóa, tùy chỉnh
độ dày nét bút.

Lưu lại tác phẩm.

DỰ ÁN

Đây là một **ứng dụng vẽ tranh đồ họa bằng Python** sử dụng thư viện Tkinter

Ứng dụng giúp học sinh:

- Hiểu nguyên lý hoạt động của tọa độ trên màn hình.
- Làm quen với việc bắt sự kiện (nhấp chuột, rê chuột).
- Phát huy tư duy logic và khả năng sáng tạo nghệ thuật.

Tính năng chính:

- **Vẽ tự do** trên khung giấy trắng bằng chuột.
- Tự do **chọn màu sắc** từ bảng màu đa dạng.
- Sử dụng **cục tẩy** và tính năng **xóa toàn bộ** trang giấy.
- Điều chỉnh **kích thước nét bút** (to/nhỏ) bằng thanh trượt.

Đây là một dự án thú vị giúp các bạn biến những dòng code khô khan thành một "xưởng họa" của riêng mình.



XỬ LÝ SỰ KIỆN (EVENT)

- Trong lập trình giao diện, "Sự kiện" (Event) là những hành động mà người dùng tương tác với máy tính, **ví dụ như:** (*nhấp chuột, kéo chuột, hoặc gõ phím*).
- Để phần mềm "hiểu" và "phản ứng" lại các hành động này, chúng ta sử dụng **hàm bind()** (gắn kết).
- Hàm này đóng vai trò như một người, **liên tục theo dõi chuột** và **gọi các đoạn code tương ứng** ra để chạy khi cần thiết.

```
canvas.bind("<Button-1>", start_draw)  
canvas.bind("<B1-Motion>", draw)
```

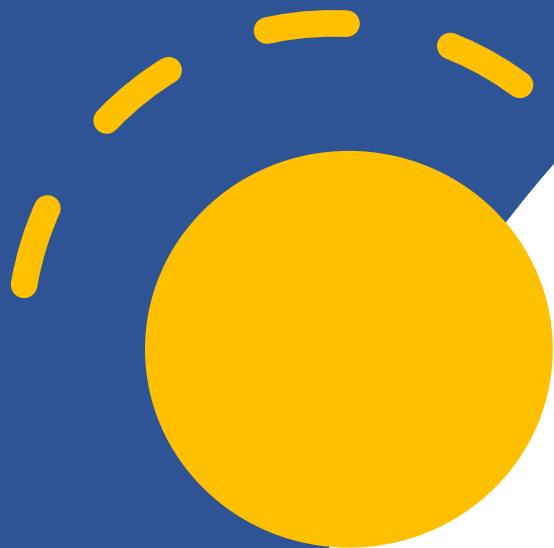
`canvas.bind("<Tên_Sự_Kiện>", tên_hàm_xử_lý)`

<Button-1>: Kích hoạt khi người dùng **nhấp chuột trái**.

Trong dự án, sự kiện này dùng để ghi nhớ tọa độ lúc vừa đặt bút xuống (`start_draw`).

<B1-Motion>: Kích hoạt khi người dùng **giữ chặt chuột trái và rê đi**. Sự kiện này chạy liên tục để nối các tọa độ lại thành nét vẽ liền mạch (`draw`).

Lưu ý: Chữ "B1" là viết tắt của Button 1 (Chuột trái).



LẬP TRÌNH DỰ ÁN

ỨNG DỤNG PAINT MINI

TẠO CỬA SỐ CHƯƠNG TRÌNH

- Thêm thư viện tkinter
- Thêm thư viện colorchooser
- Thêm thư viện filedialog

Viết chương trình hiển thị giao diện:

- Tên cửa sổ: root
- Tiêu đề: "🎨 Paint Mini"
- Kích thước: **900x600**

```
import tkinter as tk
from tkinter import colorchooser, filedialog
```

```
# ===== Cửa sổ =====
root = tk.Tk()
root.title("🎨 Paint Mini")
root.geometry("900x600")
```

```
root.mainloop() Hiển thị cửa sổ chương trình
```

ỨNG DỤNG PAINT MINI

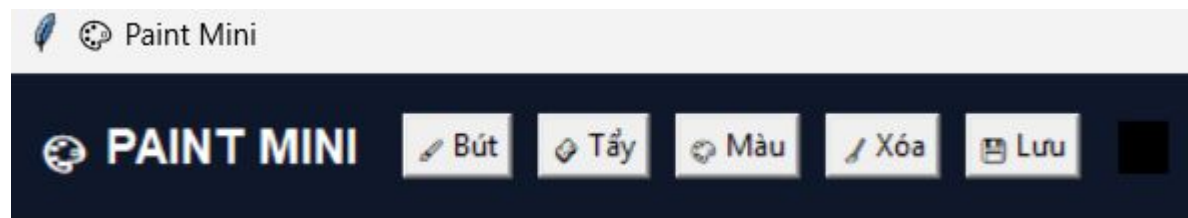
TẠO KHUNG CHỨA TIÊU ĐỀ VÀ CHỨC NĂNG

- Frame (khung) nằm trong cửa sổ chương trình (root)
- Background: Tùy chọn (ví dụ: gray, "#0f172a",...)
- Hiển thị: kéo dẫn theo hướng ngang ngang (theo trục x)

TIÊU ĐỀ ỨNG DỤNG

Tạo Label:

- Nội dung: **"PAINT MINI"**
- Font: Arial, 14, in đậm
- Màu chữ: Trắng
- Căn trái



```
top = tk.Frame(root, bg = "gray")
top.pack(fill = "x")
```

```
tk.Label(top, text="🎨 PAINT MINI",
        bg="#0f172a", fg="white",
        font=("Arial", 14, "bold")).pack(side="left", padx=10)
```

ỨNG DỤNG PAINT MINI

TẠO CANVAS (NƠI VẼ)

- Tạo biến: canvas
- Gọi hàm canvas từ Tkinter, nằm trong cửa sổ root.
- Background (màu nền): Màu trắng .
- Kéo dẫn canvas phủ toàn bộ cửa sổ : fill = "both".
- Tự động co giãn khi cửa sổ được thu nhỏ hoặc phóng to: expand = True.

```
# ===== Canvas =====
canvas = tk.Canvas(root, bg="white")
canvas.pack(fill="both", expand=True)
```

ỨNG DỤNG LUYỆN GỖ BÀN PHÍM

CÁC BIẾN TOÀN CỤC

Last_x, last_y = 0,0

Tool = "pen"

Size = 5

Color = "black"

□ Mặc định:

Công cụ là: "bút"

Độ dày nét mực là: 5

Màu là: Đen

Biến	Ý nghĩa
Last_x, last_y	Lưu lại toạ độ x,y gần nhất của con trỏ chuột.
Tool	Biến xác định công cụ đang sử dụng 'Pen' hay là 'eraser'
Size	Biến lưu độ dày nét vẽ
Color	Lưu màu sắc nét vẽ

```
# ===== Biến =====
color = "black"
size = 5
tool = "pen"
last_x, last_y = 0, 0
```

ỨNG DỤNG PAINT MINI

XỬ LÝ SỰ KIỆN (HÀNH ĐỘNG VẼ)

- Sử dụng phương thức **bind()** của **canvas**: Dùng để kết nối một hành động của người dùng với 1 chức năng của phần mềm.
- Sự kiện "<Button-1>": là hành động nhấp chuột trái.
- Sự kiện "<B1-Motion>": là hành động giữ chuột trái và kéo rê đi.
- Start_draw, draw: Hai hàm/ chức năng sẽ được thực hiện khi sự kiện xảy ra (giải thích ở slide sau).

```
canvas.bind("<Tên_Sự_Kiện>", tên_hàm_xử_lý)
```

```
canvas.bind("<Button-1>", start_draw)
canvas.bind("<B1-Motion>", draw)
```

Chú ý: "<Button-1>" và "B1-Motion" là mã sự kiện được quy định sẵn của Tkinter.

ỨNG DỤNG PAINT MINI

XỬ LÝ SỰ KIỆN (HÀNH ĐỘNG VẼ)

- **Hàm `start_draw()`:** Ghi nhớ lại **điểm xuất phát trước khi** người dùng **bắt đầu kéo chuột để vẽ**. Như cho hành động đặt ngòi bút xuống giấy.
- **`event`:** là tham số của hàm đại diện cho sự kiện nhấp chuột trái (tham số sẽ chứa thông tin vị trí nơi người dùng nhấp chuột).
- **`Last_x = event.x` và `last_y = event.y`:** Lấy thông tin vị trí ở nơi sự kiện xảy ra (tức vị trí đầu tiên chuột click trên canvas).

```
def start_draw(event):  
    global last_x, last_y  
    last_x = event.x  
    last_y = event.y
```

Chú ý: sử dụng từ khoá **global** khi khai báo hai biến `last_x`, `last_y` để cùng sử dụng chung với biến toàn cục trong suốt chương trình. Tránh lỗi khi vẽ.

ỨNG DỤNG PAINT MINI

XỬ LÝ SỰ KIỆN (HÀNH ĐỘNG VẼ)

- **Hàm draw():** hàm chứa thông tin vị trí của chuột tại khoảng khắc hiện tại (khi chuột đang kéo đi).
- **event:** là tham số của hàm đại diện cho sự kiện giữ chuột trái và rê đi (chứa vị trí hiện tại của chuột).
- Biến **draw_color:** (vì trong tkinter không có hàm nào cụ thể để tẩy), nên ta xác định **draw_color** là **màu trắng** khi **tool** là **eraser** do nền của canvas ta đã chọn là màu trắng. **Nếu tool không phải eraser** thì sẽ xác định màu được chọn của biến **color**.

```
def draw(event):
    global last_x, last_y

    draw_color = "white" if tool == "eraser" else color

    canvas.create_line(last_x, last_y, event.x, event.y,
                       fill=draw_color,
                       width=size,
                       capstyle="round")

    last_x = event.x
    last_y = event.y
```

ỨNG DỤNG PAINT MINI

XỬ LÝ SỰ KIỆN (HÀNH ĐỘNG VẼ)

- **Lệnh create_line():** dùng để vẽ một đoạn thẳng nối giữa 2 điểm.

Các tham số của create_line():

- Last_x, last_y: điểm xuất phát (tọa độ ở khoảng khắc trước đó)
- Event.x , event.y điểm kết thúc (tọa độ nơi chuột vừa rê tới)
- Fill : gán cho biến draw_color để xác định màu.
- Width: gán cho size để xác định độ dày cho nét mực.
- Capstyle: chọn "round", giúp bo tròn 2 đầu đoạn thẳng.

```
def draw(event):
    global last_x, last_y

    draw_color = "white" if tool == "eraser" else color

    canvas.create_line(last_x, last_y, event.x, event.y,
                       fill=draw_color,
                       width=size,
                       capstyle="round")

    last_x = event.x
    last_y = event.y
```

Last_x = event.x và last_y = event.y: Sau khi vẽ xong một đoạn cực ngắn, ta phải ngay lập tức lấy **vị trí hiện tại**.

ỨNG DỤNG PAINT MINI

TẠO BUTTON VÀ CHỨC NĂNG CỦA BUTTON

1. Bút (pen)

Giao diện button:

Top : hiển thị tại frame đã được tạo trước đó.

Text: chữ hiện trên button.

Command: Xác định button sẽ sử dụng hàm set_pen()

Side: căn bên trái của frame

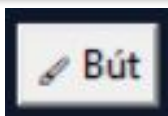
Padx: tạo khoảng trắng 2 bên cạnh của button.

Chức năng button

```
def set_pen():
    global tool
    tool = "pen"
```

Xác định cho chương trình rằng tool (công cụ) là **'pen'**. Lưu vào biến toàn cục.

Giao diện button



```
tk.Button(top, text="✎ Bút", command=set_pen).pack(side="left", padx=5)
```

ỨNG DỤNG PAINT MINI

TẠO BUTTON VÀ CHỨC NĂNG CỦA BUTTON

2. Tẩy (eraser)

Giao diện button:

Top : hiển thị tại frame đã được tạo trước đó.

Text: chữ hiện trên button.

Command: Xác định button sẽ sử dụng hàm set_eraser()

Side: căn bên trái của frame

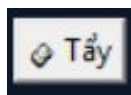
Padx: tạo khoảng trắng 2 bên cạnh của button.

Chức năng button

```
def set_eraser():
    global tool
    tool = "eraser"
```

Xác định cho chương trình rằng tool (công cụ) là **'eraser'**. Lưu vào biến toàn cục.

Giao diện button



```
tk.Button(top, text="🧽 Tẩy", command=set_eraser).pack(side="left", padx=5)
```

ỨNG DỤNG PAINT MINI

TẠO BUTTON VÀ CHỨC NĂNG CỦA BUTTON

3. Chọn màu (pick color)

Giao diện button:

Top : hiển thị tại frame đã được tạo trước đó.

Text: chữ hiện trên button.

Command: Xác định button sẽ sử dụng hàm set_eraser()

Side: căn bên trái của frame

Padx: tạo khoảng trắng 2 bên cạnh của button.

Giao diện button



```
tk.Button(top, text="🎨 Màu", command=pick_color).pack(side="left", padx=5)
```

ỨNG DỤNG PAINT MINI

TẠO BUTTON VÀ CHỨC NĂNG CỦA BUTTON

3. Chọn màu (pick color)

Giao diện ô hiển thị màu đang chọn.

Dùng hàm `label()` của tkinter với tham số:

- `Top`: đặt nhãn xác định màu lên frame.
- `bg = color`: hiển thị màu nền giống với màu của nét bút.
- `Width`: độ rộng của nhãn.

Giao diện button



```
color_box = tk.Label(top, bg=color, width=2)
color_box.pack(side="left", padx=10)
```



HELLO

ỨNG DỤNG PAINT MINI

TẠO BUTTON VÀ CHỨC NĂNG CỦA BUTTON

3. Chọn màu (pick color)

Chức năng Button:

Colorchooser.askcolor(): Gọi bảng màu để chọn.

[1]: trả về mã màu (mã hex)

□ Màu đã chọn sẽ được lưu vào biến tạm thời c.

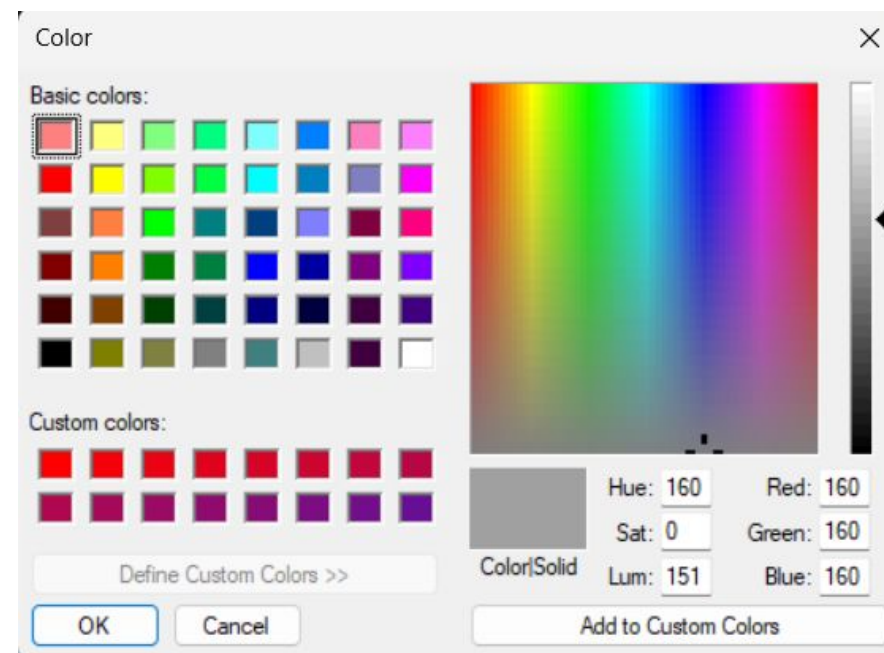
Chú ý: Thêm điều kiện if xác định biến c có tồn tại giá trị không, Tránh lỗi biến bị rỗng.

Nếu người dùng chọn màu và nhấn OK, gán màu đã chọn vào biến color toàn cục.

Cập nhập vào ô xác định màu với màu mới vừa chọn.

Chức năng button

```
def pick_color():
    global color
    c = colorchooser.askcolor()[1]
    if c:
        color = c
        color_box.config(bg=color)
```



ỨNG DỤNG PAINT MINI

TẠO BUTTON VÀ CHỨC NĂNG CỦA BUTTON

4. Xoá canvas (clear)

Giao diện button:

Tương tự các button khác (chỉ thay text = "xoá").

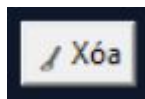
Chức năng button:

Phương thức delete của canvas: Chọn "all".

Chức năng button

```
def clear():
    canvas.delete("all")
```

Giao diện button



```
tk.Button(top, text="✎ Xóa", command=clear).pack(side="left", padx=5)
```


ỨNG DỤNG PAINT MINI

Chức năng button

TẠO BUTTON VÀ CHỨC NĂNG CỦA BUTTON

5. Lưu file (save)

Giao diện button:

Tương tự các button khác (chỉ thay text = "lưu").

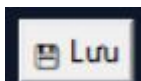
Chức năng button:

Hàm **asksaveasfilename()** của **thư viện filedialog**: với định dạng tệp (defaultextension) là .ps

Hàm **postscript()**: đóng gói thành file.

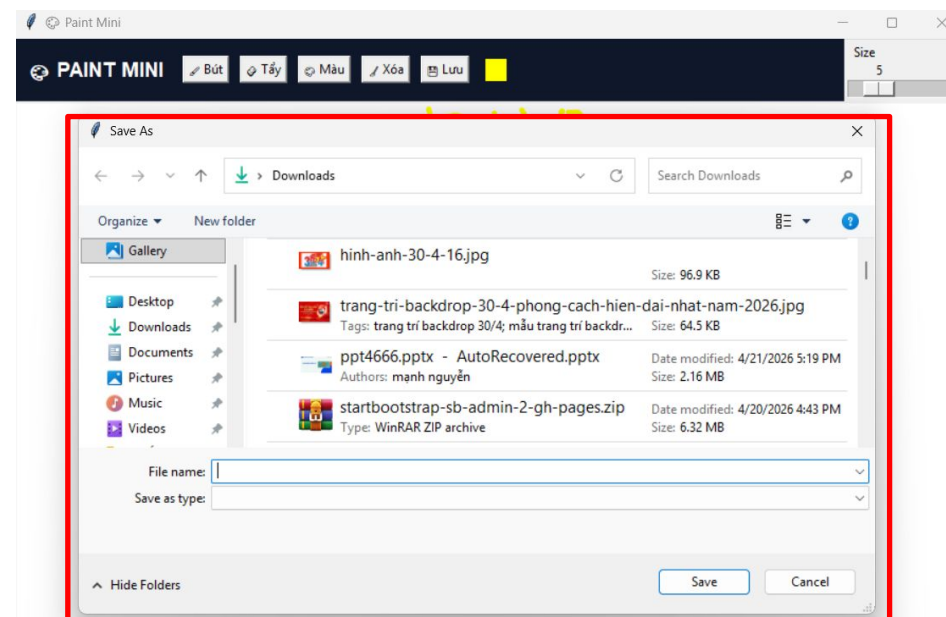
file = file là lưu theo đường dẫn đã chọn tại cửa sổ lưu file.

Giao diện button



```
tk.Button(top, text="💾 Lưu", command=save).pack(side="left", padx=5)
```

```
def save():  
    file = filedialog.asksaveasfilename(defaultextension=".ps")  
    if file:  
        canvas.postscript(file=file)
```



ỨNG DỤNG PAINT MINI

TẠO THANH TRƯỢT VÀ CHỨC NĂNG CỦA THANH TRƯỢT

6. Kích cỡ (size)

Giao diện thanh trượt:

Giá trị kích cỡ: từ 1 (from_=1) cho đến 20 (to = 20)

Hướng: theo chiều ngang (horizontal)

Command: gọi hàm chang_size.

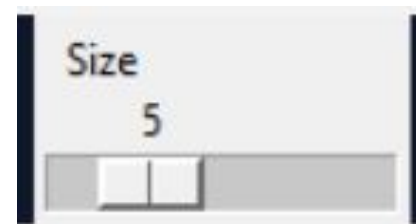
Chức năng thanh trượt:

Tham số (value) : lưu lại giá trị trên thanh trượt.

Size = int (value) : cập nhật size với giá trị mới.

Chức năng thanh trượt

```
def change_size(value):
    global size
    size = int(value)
```



Giao diện thanh trượt

```
tk.Scale(top, from_=1, to=20, orient="horizontal",
        label="Size", command=change_size).pack(side="right", padx=10)
```

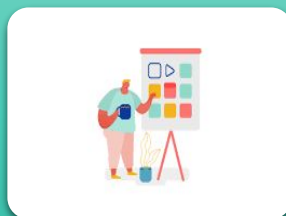
TỔNG KẾT BÀI HỌC

Thuyết minh sản phẩm



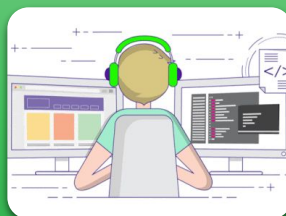
Chào hỏi

- Chào hỏi
- Giới thiệu bản thân



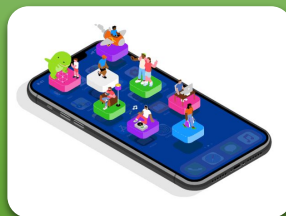
Sản phẩm

- Giới thiệu sản phẩm,
- Mô tả hoạt động



Trải nghiệm

- Trải nghiệm sản phẩm
- Thuyết minh thuật toán



Sáng tạo

- Liên hệ thực tiễn
- Đổi mới, sáng tạo

TỔNG KẾT BÀI HỌC



1. Thân thiện
2. Tự tin
3. Tích cực
4. Sáng tạo
5. Học tốt